

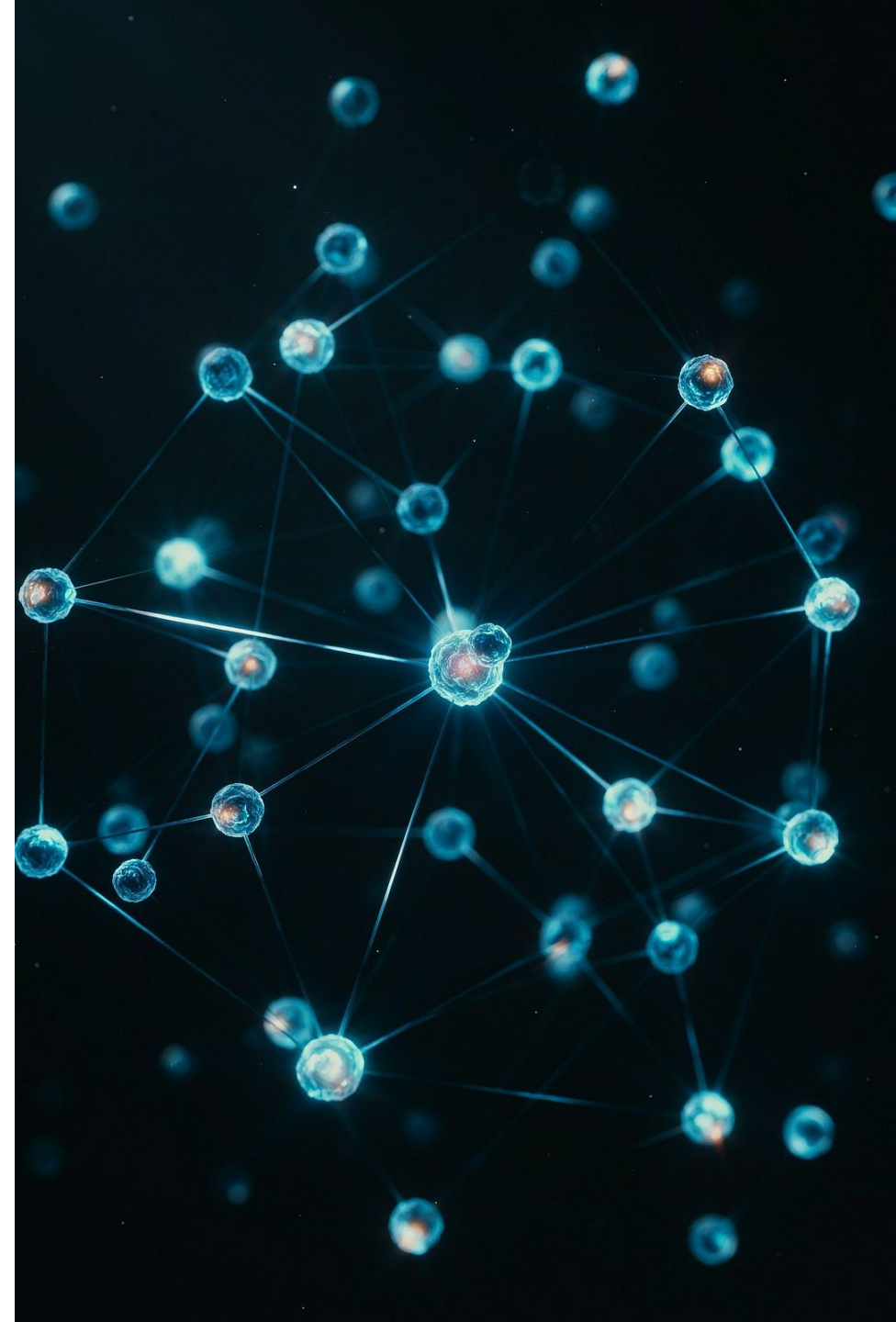
S10.2 · Redes Densas con Keras

Aprende a construir, compilar y entrenar redes neuronales completamente conectadas usando Keras sobre TensorFlow. En esta sesión cubriremos desde los fundamentos de los tensores hasta el despliegue de modelos con datos tabulares reales.

DEEP LEARNING

KERAS

TENSORFLOW



Contenido de la sesión

Un recorrido completo por los bloques esenciales para trabajar con redes densas en Keras, desde la teoría hasta la práctica.

01

Introducción a Keras y TensorFlow

El ecosistema y por qué Keras es la elección estándar.

03

Modelos: Sequential y API funcional

Dos formas de definir arquitecturas en Keras.

05

Clasificación vs. Regresión

Capas de salida y funciones de pérdida adecuadas.

02

Tensores

La estructura de datos fundamental del aprendizaje profundo.

04

Compilar, entrenar y evaluar

El ciclo completo de entrenamiento.

06

Lotes, épocas, guardado y ejemplo

Detalles prácticos para proyectos reales.

CAPÍTULO 1

Introducción a Keras y TensorFlow

¿Por qué se han convertido en el estándar de la industria para el desarrollo de modelos de aprendizaje profundo?



¿Qué es TensorFlow y qué es Keras?

TensorFlow

Biblioteca de cómputo numérico de código abierto desarrollada por Google. Permite ejecutar operaciones matemáticas sobre tensores de forma eficiente, tanto en CPU como en GPU y TPU. Es el **motor de bajo nivel** que impulsa los cálculos.

Keras

API de alto nivel integrada en TensorFlow (`tf.keras`) que simplifica enormemente la construcción de redes neuronales. Proporciona capas, optimizadores y utilidades listas para usar, siguiendo el principio de **legibilidad y rapidez de prototipado**.

- ❏ Desde TensorFlow 2.0, Keras es la interfaz oficial recomendada. No es necesario usar la API de bajo nivel para la mayoría de proyectos.

¿Por qué elegir Keras?



Rapidez de prototipado

Permite construir y probar arquitecturas en pocas líneas de código. Ideal para experimentar rápidamente con distintas configuraciones.



Legibilidad

Sintaxis clara y expresiva que hace que el código sea fácil de leer, mantener y compartir con otros miembros del equipo.



Gran comunidad

Miles de tutoriales, foros y ejemplos disponibles. La documentación oficial es extensa y está bien organizada.



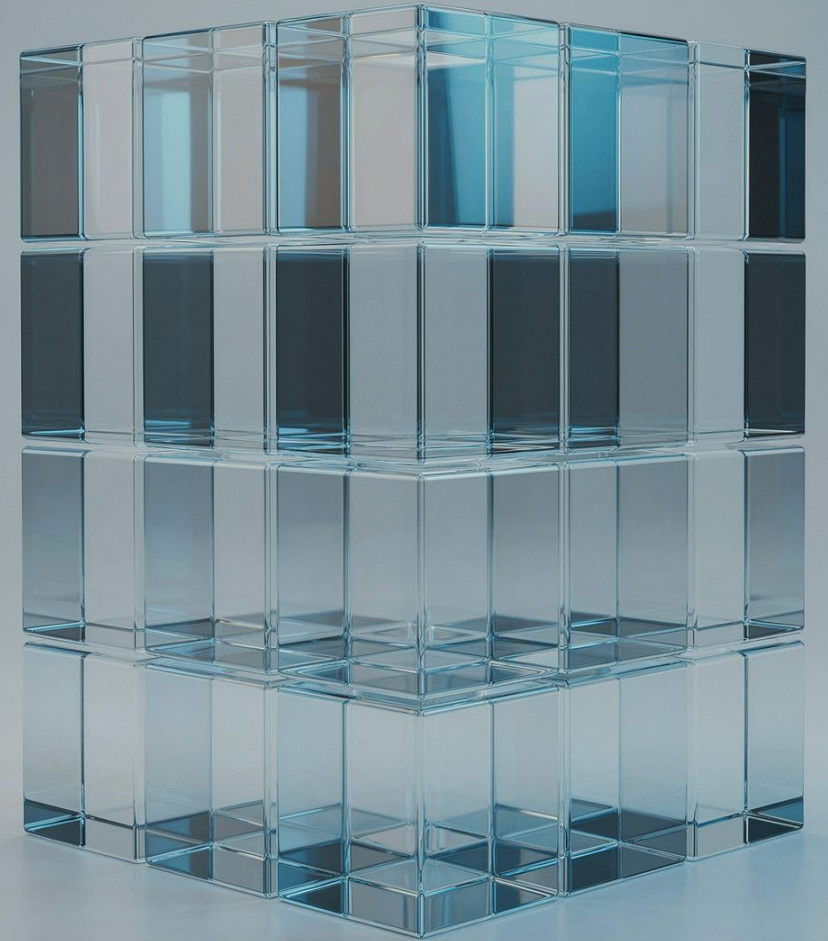
Listo para producción

Integración nativa con TensorFlow Serving, TFLite para móviles y TensorFlow.js para el navegador.

CAPÍTULO 2

Tensores: el lenguaje del aprendizaje profundo

Antes de construir modelos, es fundamental entender la estructura de datos sobre la que operan.



¿Qué es un tensor?

Un **tensor** es una generalización de escalares, vectores y matrices a cualquier número de dimensiones. En Keras, todos los datos —imágenes, textos, tablas— se representan como tensores.

1

Rango 0 — Escalar

Un único número. Por ejemplo: la pérdida 3.14.

2

Rango 1 — Vector

Una fila de números. Por ejemplo: [1.0, 0.5, 2.3].

3

Rango 2 — Matriz

Filas y columnas. Una tabla de datos o una imagen en escala de grises.

4

Rango 3 — Tensor 3D

Un bloque cúbico. Por ejemplo: imagen RGB o un lote de matrices.

```
import tensorflow as tf

# Crear un tensor desde Python
t = tf.constant([[1.0, 2.0], [3.0, 4.0]])
print(t.shape) # (2, 2)
print(t.dtype) # float32
```

Atributos clave de un tensor

Propiedades esenciales

shape

Las dimensiones del tensor, por ejemplo (1000, 28, 28, 1) para 1000 imágenes de 28×28 píxeles en escala de grises.

dtype

El tipo de dato: float32 es el más habitual en redes neuronales; int32 se usa para etiquetas enteras.

rank

El número de ejes o dimensiones. Equivalente a len(tensor.shape).

Operaciones habituales

```
x = tf.ones((3, 4))
```

```
y = tf.zeros((3, 4))
```

```
# Suma y producto
```

```
z = x + y
```

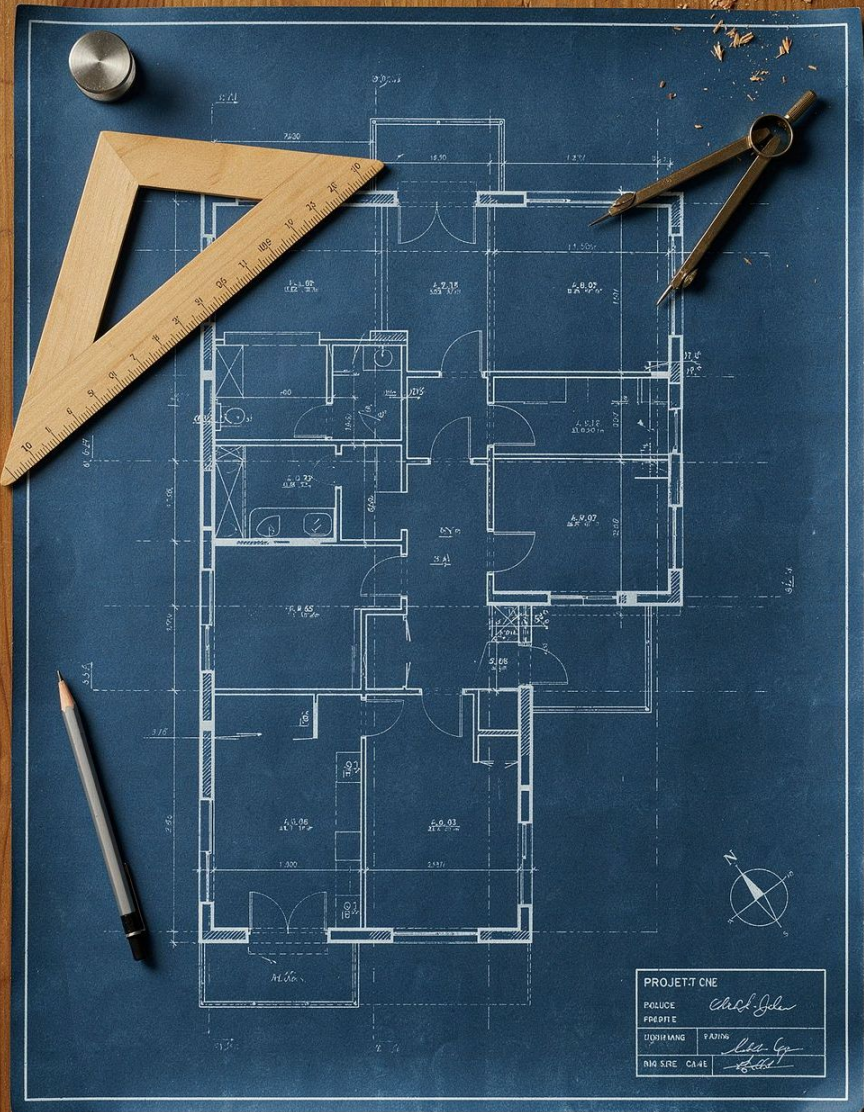
```
z = tf.matmul(x, tf.transpose(y))
```

```
# Redimensionar
```

```
r = tf.reshape(x, (12,))
```

```
# Convertir a NumPy
```

```
arr = x.numpy()
```



CAPÍTULO 3

Modelos en Keras

Keras ofrece dos formas principales de definir la arquitectura de una red neuronal: el modelo **Sequential** y la **API funcional**.

Modelo Sequential

El modelo `Sequential` es la forma más sencilla de construir una red en Keras. Las capas se apilan una tras otra, formando una **pila lineal**. Es perfecto para la mayoría de redes densas clásicas.

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(10,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
```

`Dense(64, activation='relu')`

Capa oculta con 64 neuronas y activación ReLU.

`Dense(32, activation='relu')`

Segunda capa oculta, más pequeña, para comprimir la representación.

`Dense(1, activation='sigmoid')`

Capa de salida para clasificación binaria.

API Funcional

La API funcional permite crear arquitecturas **más complejas**: múltiples entradas, múltiples salidas, conexiones residuales o ramas paralelas. Cada capa es una función que recibe y devuelve tensores.

```
inputs = keras.Input(shape=(10,))
x = layers.Dense(64, activation='relu')(inputs)
x = layers.Dense(32, activation='relu')(x)
outputs = layers.Dense(1, activation='sigmoid')(x)

model = keras.Model(inputs=inputs,
                    outputs=outputs)
```

¿Cuándo usar cada uno?

→ Sequential

Red simple, lineal, sin bifurcaciones. Ideal para aprender y para la mayoría de casos de uso.

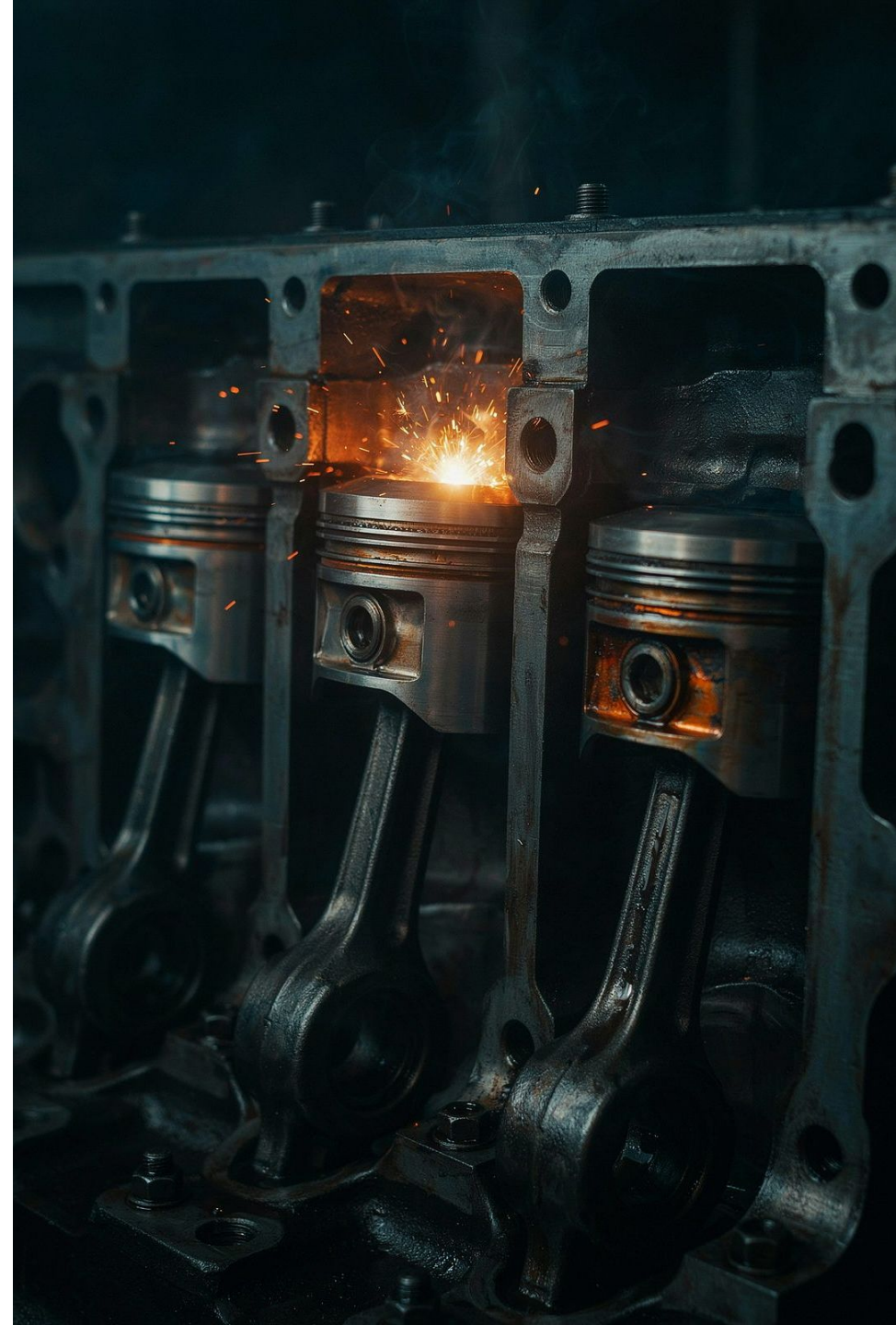
→ API Funcional

Redes con múltiples entradas/salidas, skip connections o bloques compartidos.

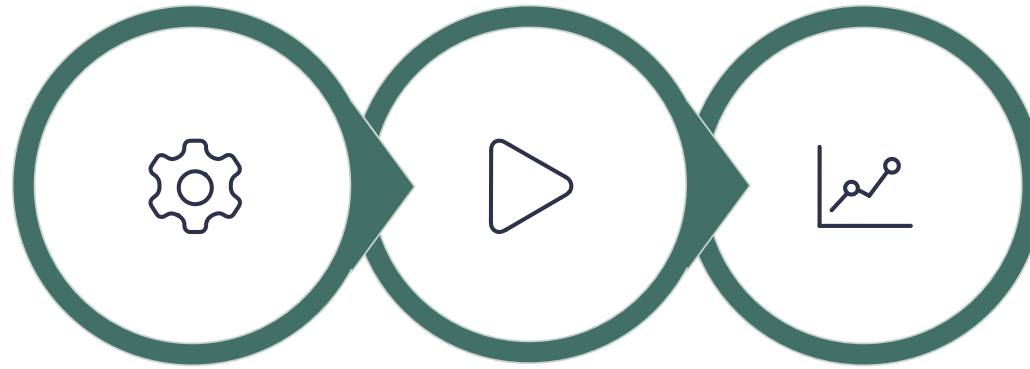
CAPÍTULO 4

Compilar y entrenar el modelo

Antes de entrenar, hay que configurar el proceso de aprendizaje mediante `compile()`. Después, `fit()` pone el motor en marcha.



El ciclo de entrenamiento en Keras



Compilar

Entrenar

Evaluar

compile()

```
model.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

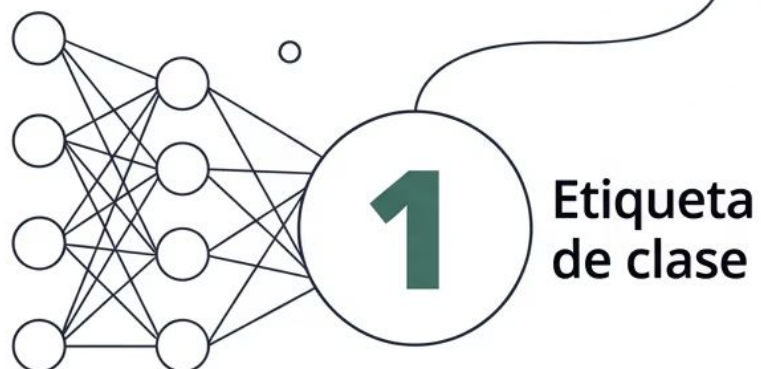
fit()

```
history = model.fit(  
    X_train, y_train,  
    epochs=20,  
    batch_size=32,  
    validation_split=0.2  
)
```

Clasificación vs. Regresión

La elección de la **capa de salida** y la **función de pérdida** es la decisión de diseño más importante al construir una red neuronal.

Clasificación



Etiqueta de clase

○ **Función de activación:**
softmax (multiclase) o
sigmoid (binaria)

◇ **Función de pérdida:**
categorical_crossentropy
o binary_crossentropy

Regresión



Número continuo

Función de activación:
Lineal (sin activación)

Función de pérdida:
mean_squared_error (MSE)
o mae (MAE)

Resumen: salidas y pérdidas según la tarea

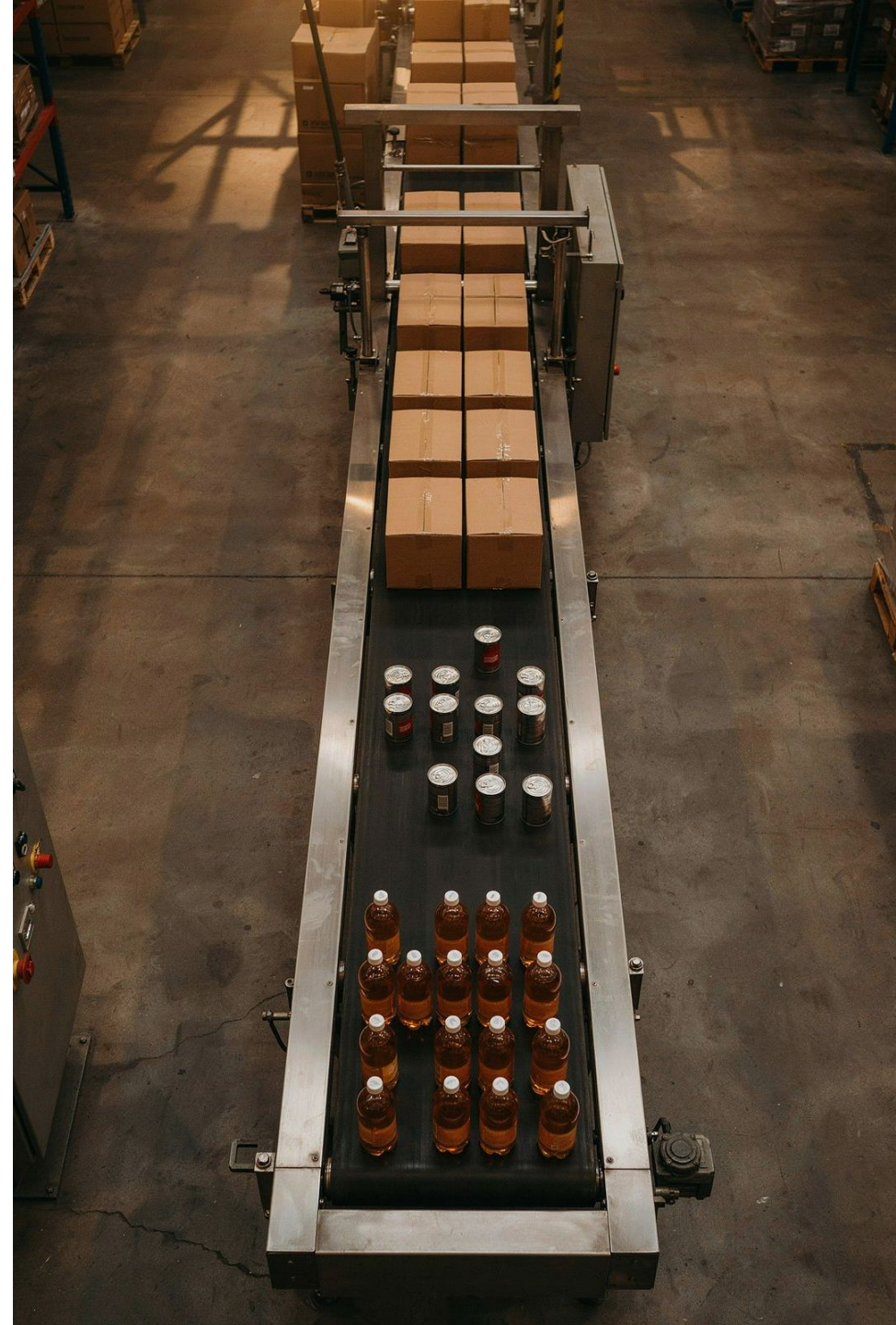
Tarea	Activación de salida	Función de pérdida	Ejemplo
Clasificación binaria	sigmoid	binary_crossentropy	Spam / no spam
Clasificación multiclase	softmax	categorical_crossentropy	Clasificar dígitos MNIST
Regresión (una salida)	Ninguna (lineal)	mse o mae	Precio de una vivienda
Regresión multisalida	Ninguna (lineal)	mse	Coordenadas de un objeto

- ❏ Elegir la pérdida incorrecta es uno de los errores más comunes. Verifica siempre que coincida con el tipo de etiquetas y la activación de salida.

CAPÍTULO 6

Lotes (Batches) y Épocas

Dos hiperparámetros fundamentales que controlan **cómo** y **cuánto** aprende la red durante el entrenamiento.



Entendiendo Batches y Epochs

Lote (Batch)

Subconjunto de datos con el que se calcula un único paso de gradiente. Un `batch_size=32` significa que el modelo actualiza sus pesos cada 32 ejemplos. Lotes más pequeños generalizan mejor; lotes más grandes son más estables y veloces.

32

`batch_size` típico

Valores habituales: 16, 32, 64, 128.

Época (Epoch)

Un recorrido completo por **todo** el conjunto de entrenamiento. Si tienes 1.000 muestras y `batch_size=32`, una época son aproximadamente 32 pasos de actualización. Más épocas permiten aprender más, pero aumentan el riesgo de sobreajuste (*overfitting*).

10–100

Epochs habituales

Depende del dataset y la tarea.

1

Paso de gradiente

Por cada lote procesado.

Guardar y Cargar Modelos

Un modelo entrenado representa horas de cómputo. Keras proporciona métodos sencillos para **persistir** y **recuperar** modelos completos o solo sus pesos.

Guardar el modelo completo

```
model.save('mi_modelo.keras')
```

Guarda arquitectura, pesos y configuración del optimizador en un único fichero. **Recomendado** para la mayoría de casos.

Cargar el modelo

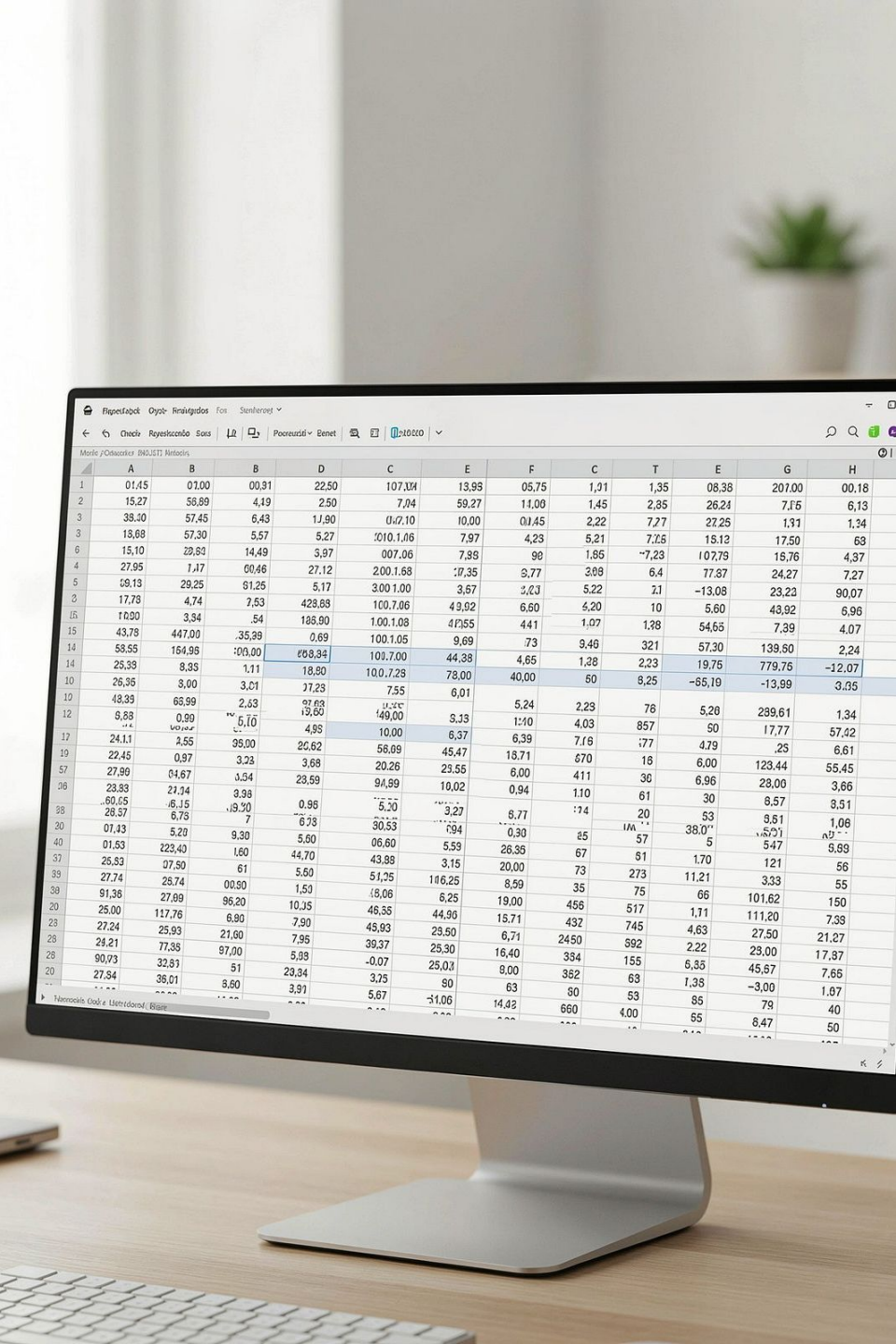
```
model =  
keras.models.load_model('mi_modelo.keras')
```

Restaura el modelo exactamente como estaba, listo para seguir entrenando o hacer predicciones.

Guardar solo los pesos

```
model.save_weights('pesos.weights.h5')  
model.load_weights('pesos.weights.h5')
```

Útil cuando ya tienes la arquitectura definida en código y solo quieres guardar los valores aprendidos.



CAPÍTULO 8

Ejemplo práctico: datos tabulares

Ponemos en práctica todo lo aprendido con un dataset tabular clásico: predecir si un paciente tiene diabetes usando el **dataset Pima Indians**.

Código completo: clasificación tabular

Un ejemplo end-to-end que integra todos los conceptos de la sesión: carga de datos, construcción del modelo, compilación, entrenamiento y evaluación.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow import keras
from tensorflow.keras import layers
```

```
# 1. Cargar y preparar datos
df = pd.read_csv('diabetes.csv')
X = df.drop('Outcome', axis=1).values
y = df['Outcome'].values
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# 2. Construir el modelo
model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
```

```
# 3. Compilar
model.compile(optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy'])
```

```
# 4. Entrenar
history = model.fit(X_train, y_train,
    epochs=50, batch_size=32,
    validation_split=0.2, verbose=0)
```

```
# 5. Evaluar
loss, acc = model.evaluate(X_test, y_test)
print(f'Accuracy en test: {acc:.2%}')
```

```
# 6. Guardar
model.save('diabetes_model.keras')
```

Conclusiones y puntos clave

- Keras simplifica el desarrollo
Su API de alto nivel permite construir redes neuronales en pocas líneas sin sacrificar flexibilidad.
- Los tensores son la base de todo
Comprender su forma (`shape`) y tipo (`dtype`) es esencial para depurar modelos.
- Salida y pérdida van de la mano
La función de activación de la capa final y la función de pérdida deben coincidir con el tipo de tarea.
- Guarda siempre tu modelo
Usa `model.save()` para no perder el trabajo de entrenamiento y facilitar el despliegue.

